

Agentic Coding Workflow: Concept and Design Plan

1. Purpose of This Document

This document explains the concept, design principles, and overall structure of a manual Agentic Coding workflow.

The purpose of this workflow is not to let an AI Agent automatically build software from a vague request. Instead, the workflow is designed to help an experienced human developer use LLM-based coding agents such as Claude Code, Codex, Anti-gravity, or similar tools in a controlled, inspectable, and repeatable software development process.

This document is intended to be given to an LLM before asking it to create, refine, or implement individual workflow skills. The LLM should use this document as the high-level design context for all later skill and prompt development work.

2. Background and Motivation

The workflow is based on the idea that Agentic Coding should be treated as structured software development orchestration, not as one-shot code generation.

A common but weak use of LLM coding is:

User gives a vague feature request
→ LLM writes code directly
→ user manually fixes errors

This approach may work for small tasks, but it breaks down in larger software projects because:

- requirements are unclear;
- scope expands without control;
- the LLM makes hidden assumptions;
- previous decisions are forgotten;
- design and implementation drift apart;
- tests are added too late or not at all;
- security and privacy concerns are reviewed too late;
- documentation and handoff are weak.

The proposed workflow replaces this with a staged process:

Idea
→ Goal
→ Stakeholders and risks
→ Requirements and acceptance criteria
→ Domain model
→ Architecture and contracts

- Data design
- MVP and release slicing
- Test strategy
- Task breakdown
- Implementation prompts
- TDD implementation loop
- Review, release, and handoff
- Workflow retrospective and skill improvement

Each stage produces explicit artifacts. Those artifacts become the input for the next stage. The human developer reviews and approves key decisions at stage gates.

3. Core Concept

The core idea is:

Agentic Coding = structured development workflow + reusable skills + human approval gates + traceable artifacts

A Skill is not merely a prompt. In this workflow, a Skill is a reusable unit of work that contains:

Skill = structured prompt + procedure + constraints + required inputs + output artifacts + review criteria

A Skill should tell the Agent:

- what stage it is operating in;
- what previous artifacts it must read;
- what it must not assume;
- what output files it must create or update;
- what uncertainties it must report;
- what decisions require human approval;
- what context must be passed to the next stage.

The workflow is intentionally manual and inspectable. The human developer decides when to run each skill, reviews the result, modifies outputs when necessary, and approves or rejects decisions before moving forward.

4. Design Goals

The workflow is designed to satisfy the following goals.

4.1 Study How Agents Work

The workflow should expose how LLM-based coding agents reason, plan, make assumptions, use context, and fail. Each stage should make the Agent's intermediate outputs visible enough for the human developer to inspect and learn from them.

4.2 Apply SDLC Concepts Explicitly

The workflow should decompose software development into stages that correspond to recognizable SDLC activities:

- goal definition;
- requirements analysis;
- domain modeling;
- architecture planning;
- data design;
- task planning;
- implementation;
- testing;
- review;
- deployment;
- documentation;
- retrospective.

4.3 Enable Stage-by-Stage Execution

Each stage should be executable independently, but it must also receive enough context from previous stages and prepare useful context for following stages.

4.4 Support Human Review and Intervention

The workflow assumes that the human developer is experienced and remains the final decision-maker. The Agent drafts, analyzes, proposes, implements, and verifies, but the human approves important decisions.

4.5 Integrate TDD, DDD, and Security Early

Testing, domain modeling, and security should not appear only at the end. They must be integrated into the workflow from the requirements, domain, architecture, and task-planning stages.

4.6 Make Results Reusable Across Projects

The workflow should produce reusable Skill prompts, artifact templates, context files, and stage gates that can be adapted to many software projects.

5. Fundamental Principles

The workflow follows these principles.

1. The Agent is not an autonomous developer. It is a structured development assistant.
2. The human developer is the final decision-maker.
3. Every stage must produce explicit artifacts.
4. Every stage must update or prepare context for the next stage.
5. Requirements must be connected to acceptance criteria.
6. Implementation tasks must be test-first or at least test-aware.
7. DDD is treated as domain language, rules, boundaries, and invariants, not merely database modeling.
8. Security, privacy, and operational constraints must be considered early.
9. Every implementation task must be traceable to requirements, domain concepts, tests, and evidence.
10. Agent outputs are proposals until approved by the human developer.
11. Assumptions must never be silently converted into decisions.
12. Rejected options must be recorded so they do not reappear later.
13. Each project should improve the workflow and the skills themselves.

A key rule is:

Agent proposal $\neq$ approved decision
Agent inference $\neq$ verified fact
Agent assumption $\neq$ requirement
Agent draft $\neq$ final artifact

6. Overall Workflow Structure

The workflow is organized into 14 stages, numbered from Stage 0 to Stage 13.

- [0] Project Intake / Existing Context Review
- [1] Service Goal Definition
- [2] Stakeholder & Risk Framing
- [3] Requirements & Acceptance Criteria
- [4] Domain Modeling / DDD
- [5] Architecture & Technical Contracts
- [6] Data Design
- [7] MVP Scope & Release Slicing
- [8] Test Strategy & Validation Harness
- [9] Task Breakdown
- [10] Implementation Prompt Writing
- [11] TDD Implementation Loop
- [12] Review / Security / Release / Handoff
- [13] Workflow Retrospective & Skill Improvement

The high-level flow is:

Understand context
→ define purpose
→ identify users and risks
→ define verifiable requirements

- model the domain
 - design architecture and contracts
 - design data structures and access rules
 - slice the MVP and releases
 - define validation strategy
 - break work into tasks
 - write implementation prompts
 - implement through a TDD loop
 - review, release, and document
 - improve the workflow
-

7. Stage-by-Stage Design

Stage 0. Project Intake / Existing Context Review

Purpose

Identify whether the project is greenfield, brownfield, a prototype, a research tool, or an extension of an existing system. The Agent must not start requirements or architecture work before understanding the current context.

Typical Skills

project-intake
codebase-context-review

Key Questions

Is this a new project or an existing codebase?
What technology stack is already fixed?
What documents, code, tests, and deployment settings already exist?
What areas must not be changed?
What decisions has the user already made?

Artifacts

00_project_intake.md
00_existing_context_review.md
context_packet.md

Human Gate

The human developer approves:

project type
fixed technology stack
existing documents and code to use
forbidden change areas
initial constraints

Stage 1. Service Goal Definition

Purpose

Define why the service should exist before deciding what to build.

Typical Skill

service_goal_definition

Artifacts

01_service_goal.md

Required Content

problem definition
target users
core value
success criteria
non-goals
initial assumptions
open questions

Human Gate

The human developer approves:

service goal
primary users
success criteria
non-goals

Stage 2. Stakeholder & Risk Framing

Purpose

Identify users, operators, administrators, external systems, regulatory concerns, security risks, and privacy risks early.

Typical Skills

stakeholder-analysis
early-security-privacy-screening

Artifacts

02_stakeholders.md
02_risk_privacy_screening.md

Required Content

stakeholders
user roles
permission levels
sensitive data
personal data
external API or LLM data transfer
administrator powers
audit log needs
initial security risks

Human Gate

The human developer approves:
role and permission direction
personal or sensitive data handling direction
external API or LLM transfer policy
initial risk assumptions

Stage 3. Requirements & Acceptance Criteria

Purpose

Convert goals into functional requirements, non-functional requirements, scenarios, edge cases, and acceptance criteria.

This is where TDD begins conceptually. The Agent does not write code here, but it must produce requirements that can later be turned into tests.

Typical Skills

requirements-decomposition
acceptance-criteria-writer

Artifacts

03_requirements.md
03_acceptance_criteria.md
03_traceability_matrix.md

Requirement Format

Requirement ID:
User Role:
Goal:
Functional Description:
Acceptance Criteria:
Edge Cases:
Related Non-Functional Requirements:
Security / Privacy Conditions:
Validation Method:

Human Gate

The human developer approves:

must-have requirements
acceptance criteria
non-scope items
whether requirements are testable

Stage 4. Domain Modeling / DDD

Purpose

Model the domain before designing the database or implementation structure. This stage applies Domain-Driven Design principles.

DDD in this workflow means:

ubiquitous language
core domain concepts

entities
value objects
aggregates
invariants
state transitions
domain events
bounded contexts
context map

It does not mean simply extracting database tables.

Typical Skills

domain-modeling
ddd-bounded-context-modeling

Artifacts

04_ubiquitous_language.md
04_domain_model.md
04_bounded_contexts.md
04_business_rules_invariants.md
04_domain_events.md

Important Rules

Do not design database tables first.
Define domain concepts and business rules first.
Treat aggregates as consistency boundaries, not just data containers.
Connect invariants and state transitions to future tests.

Human Gate

The human developer approves:

domain terms
entities and value objects
aggregate boundaries
bounded contexts
business rules

Stage 5. Architecture & Technical Contracts

Purpose

Translate the domain model and requirements into system structure, module boundaries, API contracts, integration boundaries, authentication, authorization, and data flow.

Typical Skills

architecture-planning
api-contract-design
integration-boundary-design

Artifacts

05_architecture_plan.md
05_module_boundaries.md
05_api_contracts.md
05_integration_contracts.md
05_architecture_decisions.md

Required Content

architecture style
frontend/backend boundary
module boundaries
API contracts
external integrations
authentication and authorization flow
error handling policy
logging and observability policy
major trade-offs

Human Gate

The human developer approves:

architecture direction
major technical choices
API contracts
authorization model
external integration approach

Stage 6. Data Design

Purpose

Design data structures after domain and architecture decisions are clear. Data design is separated from domain modeling and from implementation.

Typical Skills

data-modeling
database-schema-design
database-security-rule-design
migration-planning

Artifacts

06_conceptual_data_model.md
06_logical_schema.md
06_physical_schema.md
06_indexes.md
06_migration_plan.md
06_data_security_rules.md

Required Content

conceptual data model
logical schema
physical schema
indexes
query patterns
migration plan
data retention and deletion policy
security rules
access control

Human Gate

The human developer approves:

schema
data ownership
security rules
migration strategy

Stage 7. MVP Scope & Release Slicing

Purpose

Prevent uncontrolled scope expansion. Decide what belongs in the MVP, what belongs in later releases, and what is explicitly out of scope.

Typical Skills

mvp-scope-planning
release-slicing

Artifacts

07_mvp_scope.md
07_release_slices.md
07_out_of_scope.md

Scope Categories

Must
Should
Could
Won't / Non-scope
Later

Human Gate

The human developer approves:

MVP scope
out-of-scope items
release order

Stage 8. Test Strategy & Validation Harness

Purpose

Define the validation structure before implementation begins. This stage makes TDD and test-aware implementation practical.

Typical Skills

test-strategy
acceptance-test-design
validation-harness-planning

Artifacts

08_test_strategy.md
08_acceptance_tests.md
08_validation_commands.md
08_manual_test_plan.md

Required Content

unit test targets
integration test targets
E2E test targets
acceptance tests
fixture and test data strategy
manual verification steps
commands to run
pass/fail criteria
CI applicability

Traceability Rule

Every requirement and task should be connected to at least one validation method.

Requirement → Acceptance Criteria → Test Case → Task → Implementation Evidence

Human Gate

The human developer approves:

test strategy
manual validation criteria
automated test scope

Stage 9. Task Breakdown

Purpose

Convert the approved design into small development tasks that an Agent can safely execute.

Typical Skills

task-breakdown
task-card-generator
dependency-ordering

Artifacts

09_task_inventory.md
09_task_cards.md
09_dependency_order.md
09_traceability_matrix.md

Task Card Format

Task ID:
Title:
Linked Requirement:
Linked Domain Concept:
Linked API / Module:
Change Scope:
Files likely to change:
Dependencies:
Acceptance Criteria:
Required Tests:
Manual Verification:
Security/Privacy Notes:
Definition of Done:
Rollback / Recovery Notes:

Human Gate

The human developer approves:

task size
task order
definition of done for each task

Stage 10. Implementation Prompt Writing

Purpose

Convert task cards into executable prompts for coding agents.

Typical Skill

implementation-prompt-writer

Artifacts

10_implementation_prompts.md
10_prompt_handoff_packets.md

Required Prompt Structure

Each implementation prompt must include:

1. Goal
2. Context
3. Approved Inputs
4. Files to inspect first
5. Allowed change scope
6. Forbidden changes
7. Required implementation steps
8. Required tests
9. Commands to run
10. Expected evidence
11. Output files to update
12. Next context_packet.md update instructions

Important Rule

A prompt must not merely say “build this feature.” It must define:

what to change
what not to change
how to verify the change
what tests to run
what evidence to report

Human Gate

The human developer approves:

implementation prompts
allowed change scope
execution order

Stage 11. TDD Implementation Loop

Purpose

Implement the system task by task. This is not one large implementation stage. It is a repeated loop over approved task cards.

Typical Skills

frontend-implementation
backend-implementation

database-migration-implementation
test-first-implementation

Per-Task Loop

1. Read context.
2. Inspect existing code.
3. Restate the task.
4. Write or identify a failing test.
5. Run the test and confirm failure.
6. Implement the minimal change.
7. Run relevant tests.
8. Refactor.
9. Run broader regression tests.
10. Record evidence.
11. Update result.md.
12. Update context_packet.md.

Artifacts

11_task_result_{task_id}.md
11_test_evidence_{task_id}.md
updated_context_packet.md

Result Format

Task ID:
Summary:
Files Changed:
Tests Added:
Tests Run:
Test Result:
Known Limitations:
Assumptions Used:
Open Questions:
Follow-up Tasks:

TDD Rule

The implementation skill should include this instruction:

Do not implement the feature before identifying or writing the test that will verify it.

If a true test-first approach is not feasible, explain why and create a test-aware validation plan before implementation.

Human Gate

The human developer reviews:

- code change scope
- test evidence
- requirement satisfaction
- unnecessary feature additions

Stage 12. Review / Security / Release / Handoff

Purpose

Perform final review, security/privacy review, release readiness check, deployment planning, operations preparation, and documentation handoff.

Security and testing should already have been considered earlier. This stage is the final verification gate, not the first time security and testing appear.

Typical Skills

- code-review
- security-privacy-review
- deployment-operations
- documentation-handoff

Artifacts

- 12_code_review.md
- 12_security_privacy_review.md
- 12_release_readiness.md
- 12_deployment_plan.md
- 12_operations_runbook.md
- 12_documentation_handoff.md

Review Items

- requirements satisfaction
- test results
- code quality
- performance issues
- security issues
- privacy issues
- environment variables
- deployment procedure

rollback procedure
documentation freshness

Human Gate

The human developer approves:

release
deployment
operations responsibility
documentation handoff

Stage 13. Workflow Retrospective & Skill Improvement

Purpose

The project goal is not only to build software but also to study agents and improve the workflow. Therefore, the workflow itself must be reviewed after each project or major release.

Typical Skills

workflow-retrospective
skill-improvement-planning

Artifacts

13_workflow_retrospective.md
13_skill_improvement_backlog.md
13_agent_failure_patterns.md
13_reusable_lessons.md

Required Content

what the Agent did well
where the Agent repeatedly failed
which prompts were ambiguous
where human judgment was essential
what rules should be reused in future projects
which SKILL.md files should be revised
which new skills should be created
which skills should be deleted or merged

Human Gate

The human developer approves:

workflow improvements
skill revision priorities
how lessons will be applied to the next project

8. Common Stage Execution Loop

Every stage should follow the same execution protocol.

1. Read the previous context_packet.md.
2. Read only the necessary previous artifacts.
3. Restate the purpose of the current stage.
4. Identify missing information and uncertainty.
5. Produce the stage artifacts.
6. List new decision candidates.
7. List new assumption candidates.
8. List open questions.
9. Request human review.
10. Record only approved decisions in DECISIONS.md.
11. Record assumptions in ASSUMPTIONS.md.
12. Record open questions in OPEN_QUESTIONS.md.
13. Update context_packet.md for the next stage.

This loop is essential because it prevents the Agent from silently converting suggestions into facts.

9. Context Management System

The workflow uses a small set of persistent context documents.

```
/context
context_packet.md
DECISIONS.md
ASSUMPTIONS.md
OPEN_QUESTIONS.md
REJECTED_OPTIONS.md
TRACEABILITY_MATRIX.md
```

9.1 context_packet.md

context_packet.md is not a complete project history. It is the minimal operational context required by the next stage.

Recommended structure:

context_packet.md

1. Current Project State

Current stage, completed stages, next stage.

2. Approved Decisions

Only decisions explicitly approved by the human developer.

3. Working Assumptions

Temporary assumptions that are not yet verified.

4. Open Questions

Unresolved questions that may affect later stages.

5. Rejected / Superseded Options

Options that were considered but rejected or replaced.

6. Constraints That Must Not Be Violated

Technology, security, privacy, schedule, and scope constraints.

7. Key Domain / Architecture Context

Essential domain and architecture context for the next stage.

8. Required Inputs for Next Stage

Artifacts the next stage must read.

9. Do Not Do

Actions the next Agent must avoid.

9.2 DECISIONS.md

Stores only human-approved decisions.

9.3 ASSUMPTIONS.md

Stores assumptions that are useful for progress but are not yet confirmed.

9.4 OPEN_QUESTIONS.md

Stores unresolved questions that may affect requirements, design, implementation, tests, or deployment.

9.5 REJECTED_OPTIONS.md

Stores options that should not be revived unless the human explicitly reopens them.

9.6 TRACEABILITY_MATRIX.md

Maintains links between:

Goals

- Requirements
 - Acceptance Criteria
 - Domain Concepts
 - Architecture Components
 - Data Models
 - Tasks
 - Tests
 - Implementation Evidence
-

10. Recommended Project Folder Structure

A project using this workflow may use the following documentation structure.

```
/workflow
/00_intake
/01_goal
/02_stakeholders_risk
/03_requirements
/04_domain
/05_architecture
/06_data
/07_mvp_release
/08_test_strategy
/09_tasks
/10_prompts
/11_implementation_results
/12_review_release_handoff
/13_retrospective
```

```
/workflow/context
context_packet.md
DECISIONS.md
ASSUMPTIONS.md
OPEN_QUESTIONS.md
REJECTED_OPTIONS.md
TRACEABILITY_MATRIX.md
```

Each stage may also have its own folder containing:

SKILL.md
result.md
context_packet.md
review_notes.md

If a stage-specific user instruction file exists, the Agent must read it before executing the stage. For example:

USER_INSTRUCTIONS.md
review_notes.md

These files allow the human developer to provide corrections, review comments, or additional decisions without rewriting the entire prompt.

11. TDD Integration

TDD is not a separate final testing stage. It appears in three places.

11.1 Requirements Stage

Each requirement must have acceptance criteria.

Requirement → Acceptance Criteria

11.2 Task Breakdown Stage

Each task must specify required tests.

Task → Required Tests → Definition of Done

11.3 Implementation Stage

Each implementation task follows a Red-Green-Refactor style loop when feasible.

write or identify failing test
→ confirm failure
→ implement minimal code
→ pass test
→ refactor
→ run regression tests
→ record evidence

If strict test-first implementation is not feasible, the Agent must explain why and create a test-aware validation plan before coding.

12. DDD Integration

DDD is centered in Stage 4 but influences later stages.

Ubiquitous Language

- Domain Concepts
- Entities / Value Objects
- Aggregates
- Invariants
- Domain Events
- Bounded Contexts
- Module Boundaries
- API Contracts
- Data Model
- Tests

Important mappings:

- Invariant → Unit Test
- State Transition → Scenario Test
- Domain Event → Integration Test
- Bounded Context → Module Boundary
- Aggregate → Transaction Boundary

The Agent must not reduce DDD to database table extraction. The purpose of DDD is to preserve domain meaning and business rules through architecture, implementation, and tests.

13. Security and Privacy Integration

Security and privacy are handled at multiple points.

- Stage 2: early risk and privacy screening
- Stage 3: security/privacy requirements
- Stage 5: architecture-level security decisions
- Stage 6: data access and security rules
- Stage 8: validation of security-sensitive behavior
- Stage 11: task-level security notes and tests
- Stage 12: final security/privacy review

The Agent must explicitly identify:

- personal data
- sensitive data
- external data transfer
- LLM/API data exposure
- role-based access rules
- audit logging needs

data retention and deletion rules
security assumptions

14. Human Approval Gate

Human approval gates are essential. The Agent may propose, but the human approves.

The following decisions require explicit human approval:

- service goal
- MVP scope
- non-scope items
- personal data handling
- external API or LLM transfer policy
- domain terminology
- aggregates and bounded contexts
- architecture direction
- database schema
- API contracts
- release order
- implementation prompts
- deployment approval
- workflow improvement priorities

At the end of each stage, the Agent should report:

User Approval Required

Decisions to Approve

- ...

Assumptions to Confirm

- ...

Open Questions

- ...

Risks

- ...

Recommended Next Step

- ...

15. Skill Design Requirements

Each Skill created for this workflow should follow a common template.

A Skill should include:

1. Skill name
2. Stage
3. Purpose
4. When to use
5. Required inputs
6. Optional inputs
7. Files to read first
8. Human instruction files to check
9. Step-by-step procedure
10. Output artifacts
11. Required sections in result.md
12. Context packet update instructions
13. Decision / assumption / open-question handling
14. Human approval gate
15. Quality checklist
16. Failure handling
17. Do-not-do rules

A Skill must distinguish:

approved decisions
working assumptions
open questions
rejected options
recommendations

A Skill must not silently treat missing information as confirmed. If information is missing, it should record the gap in the appropriate section and proceed with a clearly marked assumption only when safe.

16. Existing Skills and Revised Placement

The original skill set can be reorganized as follows.

Skill	Revised Position	Revision Direction
service_goal_definition	Stage 1	Keep and strengthen approval criteria
stakeholder-analysis	Stage 2	Connect to risk and privacy screening

Skill	Revised Position	Revision Direction
requirements-decomposition	Stage 3	Require acceptance criteria
mvp-scope-planning	Stage 7	Combine with release slicing
domain-modeling	Stage 4	Strengthen with DDD concepts
architecture-planning	Stage 5	Include API and data contracts
database-design	Stage 6	Split conceptual/logical/physical/migration/security concerns
task-breakdown	Stage 9	Add test requirements and traceability
implementation-prompt-writing	Stage 10	Require allowed scope, forbidden changes, tests, and evidence
frontend-implementation	Stage 11	Execute inside TDD loop
backend-implementation	Stage 11	Execute inside TDD loop
test-strategy	Stage 8 and Stage 11	Use before and during implementation
code-review	Stage 12	Use final and task-level review
security-privacy-review	Stage 2 and Stage 12	Use early screening and final review
deployment-operations	Stage 12	Combine with release readiness
documentation-handoff	Stage 12	Combine with operations handoff

Additional useful skills:

project-intake
 codebase-context-review
 acceptance-criteria-writer
 ddd-bounded-context-modeling
 api-contract-design
 data-security-rule-design
 release-slicing
 validation-harness-planning
 test-first-implementation
 workflow-retrospective
 skill-improvement-planning

17. How an LLM Should Use This Document

When an LLM is asked to help develop this workflow, it should follow these rules.

1. Treat this document as the high-level workflow design.
2. Do not collapse the workflow into one generic prompt.
3. Preserve stage boundaries unless explicitly asked to simplify.
4. When creating a Skill, define its inputs, outputs, procedure, and context handoff.
5. Always distinguish human-approved decisions from Agent-generated suggestions.
6. Include human review gates in every stage.
7. Include context_packet.md update instructions in every Skill.
8. Preserve traceability from goals to requirements, tests, tasks, and implementation evidence.
9. Integrate TDD, DDD, and security/privacy as cross-cutting concerns.
10. Avoid assuming missing information. Mark it as an assumption or open question.
11. Design each Skill so that it can be executed manually by a human developer using an Agentic Coding tool.
12. Make outputs concrete markdown files, not vague summaries.

The LLM should understand that the goal is not to build one software product immediately. The goal is to build a reusable manual Agentic Coding workflow that can later be applied to many software development projects.

18. Final Summary

This workflow is a manual, inspectable, SDLC-based Agentic Coding process for experienced developers.

It transforms software development from:

vague request → direct code generation

into:

structured context

- approved goals
- verifiable requirements
- domain model
- architecture and data contracts
- scoped tasks
- implementation prompts
- TDD implementation
- review and release
- workflow improvement

The most important design decision is that the Agent does not own the software process. The human developer owns the process, and the Agent executes structured skills inside that process.

The workflow should therefore be designed around:

- clear artifacts
- explicit context handoff
- human approval gates
- testable requirements
- DDD-based domain clarity
- security and privacy awareness
- traceable implementation evidence
- continuous improvement of the skills themselves